

DeepSight

Specifiche Tecniche

Ricerca avanzata di foto e video in un archivio privato locale, con intelligenza artificiale

Titolo del progetto	DeepSight
Numero Commessa	DAI252701
Cliente	EURIX
ID Gruppo	Gruppo 3
Nome Gruppo	I Quantizzati
Data	20/07/2026
Versione	1.0

Sommario

1. Analisi della Realtà di Riferimento	4
1.1 Contesto e Obiettivo	4
2. Funzionalità dell'Interfaccia Grafica	4
2.1 Pannello Notifiche (barra laterale)	4
2.2 Dashboard	4
2.3 Galleria	5
2.4 Caricamento & Import	5
2.5 Ricerca Avanzata	6
2.6 Persone	6
2.7 Controllo Integrità Archivio	7
3. Persistenza dei Dati	7
3.1 Database Relazionale — SQLite	7
3.2 Database Vettoriale — ChromaDB	7
3.3 Doppia memorizzazione degli embedding (scelta progettuale)	8
4. Requisiti del Sistema	8
4.1 Piattaforma di Sviluppo	8
4.2 Modelli di Intelligenza Artificiale	8
4.3 Architettura Software	9
4.4 Requisiti Minimi Hardware	10
5. Descrizione delle Classi e dei Moduli	10
5.1 models.py	11
5.2 processor.py	11
5.3 chroma_store.py — Classe ChromaStore	11
5.4 database.py	11
5.5 qwen_client.py e persone.py	12
5.6 config.py	12
6. Deployment	12
6.1 Installazione (una tantum)	12

6.2 Avvio	12
6.3 Configurazione	13
6.4 Controllo di Versione	13
7. Riferimenti	13

1. Analisi della Realtà di Riferimento

1.1 Contesto e Obiettivo

DeepSight è un'applicazione desktop locale per l'archiviazione e la ricerca avanzata di foto e video in un archivio privato a cui l'utente aggiunge progressivamente nuovi contenuti. A differenza degli strumenti tradizionali, che si basano su metadati inseriti manualmente (titolo, descrizione) o già presenti nel file (data, luogo, risoluzione), DeepSight estrae automaticamente informazioni semantiche dai contenuti tramite modelli di intelligenza artificiale eseguiti interamente in locale, garantendo la totale privacy dei dati: nessun contenuto viene inviato a servizi cloud.

Il software si rivolge a tre categorie di utenti:

- Content creator — archiviazione e recupero rapido di contenuti per web, blog e piattaforme social;
- Associazioni culturali o ricreative — gestione di archivi legati a eventi e iniziative, e pubblicazione online;
- Aziende private — gestione on-premise dei propri contenuti istituzionali.

L'utente interagisce con l'applicazione esclusivamente tramite la sua interfaccia grafica, sia per il caricamento di nuovi contenuti sia per la ricerca avanzata e il download degli elementi di interesse.

2. Funzionalità dell'Interfaccia Grafica

L'interfaccia è realizzata con Streamlit. La navigazione è organizzata in quattro sezioni principali — Dashboard, Galleria, Ricerca Avanzata e Persone — raggiungibili dalla barra superiore (navbar), al cui centro campeggia il logo dell'applicazione. Sulla stessa barra un pulsante a forma di scudo apre il Controllo Integrità Archivio, mentre un pulsante flottante «+» in basso a destra apre le funzioni di caricamento.

2.1 Pannello Notifiche (barra laterale)

La barra laterale è il pannello Notifiche dell'applicazione: conserva la cronologia degli esiti delle operazioni (caricamenti, eliminazioni, controlli di integrità — mostrati anche come avvisi temporanei) e lo stato della coda di elaborazione, con conteggi per stadio, tempo stimato, pausa/ripresa e ripetizione degli elementi falliti; si aggiorna da sola ogni secondo. Il pannello si apre e chiude con il pulsante a campanella; quando è chiuso, il logo compare al centro della barra superiore. I parametri di funzionamento (dispositivo di calcolo, campionamento video, soglie) non sono esposti nell'interfaccia: risiedono nel file di configurazione (sezione 6.3).

2.2 Dashboard

Fornisce una sintesi dell'archivio tramite quattro metriche principali — Elementi Analizzati, Frame Estratti, Volti Rilevati e Dimensione Archivio — seguite dal Dettaglio Risorse (conteggio per tipo, stato di elaborazione, dimensione del database e cartella dell'archivio) e da una griglia responsive degli elementi aggiunti di recente (Figura 1).

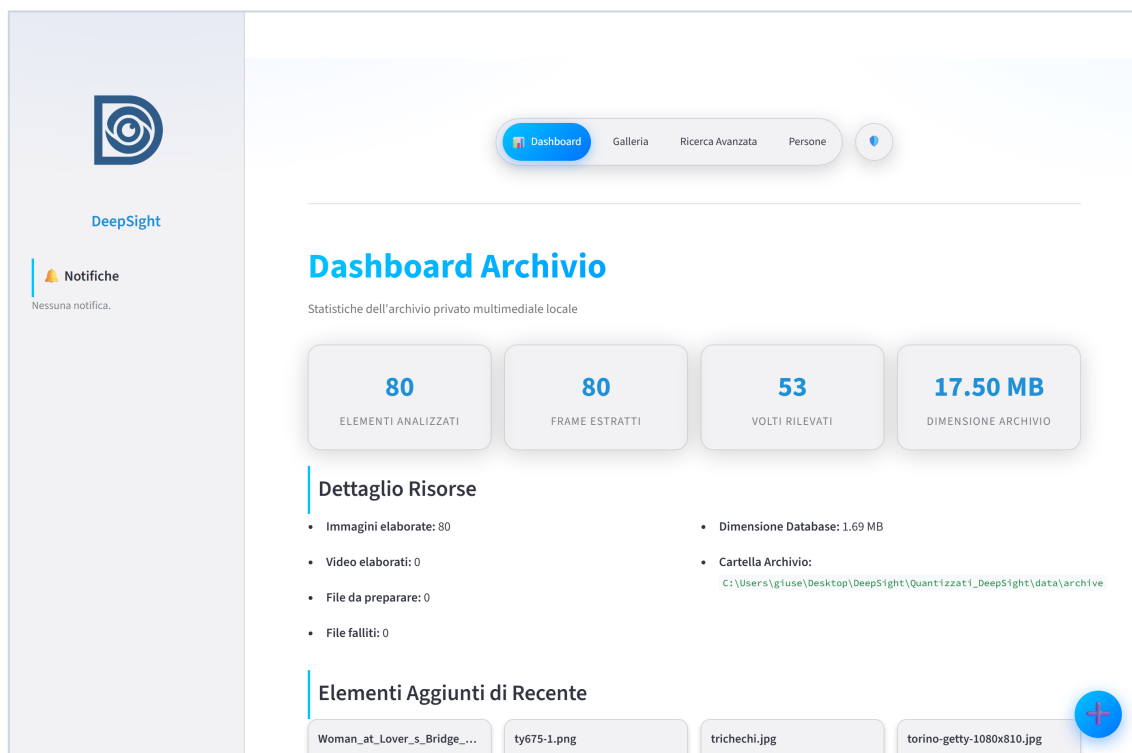


Figura 1 — La Dashboard: pannello Notifiche, metriche principali e dettaglio risorse.

2.3 Galleria

Visualizzazione a griglia responsive di tutti gli elementi dell'archivio, senza necessità di una ricerca. Offre filtro per tipo di file (Tutti / Immagini / Video), ordinamento (più recenti / meno recenti / nome) e paginazione. Ogni scheda mostra l'anteprima, i metadati e i tag rilevati, con un pannello Azioni e Dettagli per download ed eliminazione; è disponibile la selezione multipla per il download in blocco (ZIP) o l'eliminazione. Le immagini si aprono a schermo intero con un clic, mentre i video sono riproducibili direttamente in linea tramite un player.

2.4 Caricamento & Import

Accessibile dal pulsante flottante «+», offre due modalità:

- Upload diretto: selezione multipla di file dalla UI (.jpg, .jpeg, .png, .mp4, .avi, .mov, .mkv). L'elaborazione avviene in un thread in background con indicatore di avanzamento, così l'utente può continuare a navigare.
- Scansione cartella condivisa: importazione in blocco da un percorso locale specificato dall'utente.

2.5 Ricerca Avanzata

Organizzata in quattro schede (Figura 2), riassunte nella tabella seguente. Un blocco Filtri di ricerca comune (tipo, luogo, intervallo di date) è applicabile a tutte le modalità.

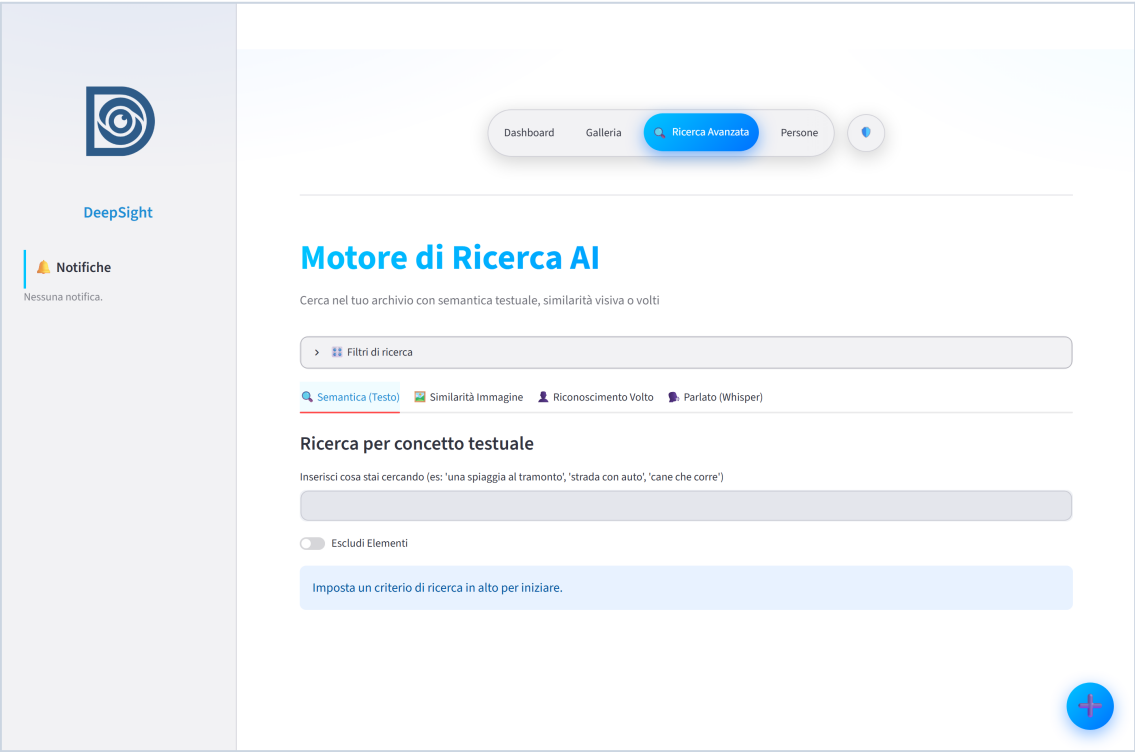


Figura 2 — La Ricerca Avanzata: le quattro modalità e i filtri comuni.

Scheda	Input	Modello AI	Descrizione
Ricerca Semantica	Testo libero	Qwen3-VL-Embedding	Confronto embedding testo→immagine nello spazio comune
Similarità Immagine	Immagine di query	Qwen3-VL-Embedding	Confronto embedding immagine→immagine
Riconoscimento Volto	Immagine con volto	MTCNN + FaceNet	Confronto embedding facciale (512-d)
Parlato	Parola chiave	Whisper	Ricerca full-text sulle trascrizioni audio dei video

Tabella 1 — Le quattro modalità della Ricerca Avanzata.

La ricerca semantica supporta anche un prompt negativo («Escludi Elementi»): il punteggio finale diventa $\cos(\text{query}, \text{img}) - \lambda \cdot \cos(\text{negativo}, \text{img})$. Le ricerche Qwen non applicano una soglia di similarità (le distribuzioni del coseno dipendono dalla query): i risultati sono deduplicati per file originale, ordinati per rilevanza e paginati (i migliori 5, poi «Mostra altri»). Solo la ricerca per volto mantiene una soglia empirica.

2.6 Persone

Raggruppa automaticamente i volti rilevati in persone: a ogni nuovo volto viene assegnato il cluster (centroide) più vicino oppure, se nessuno è abbastanza simile, ne viene creato uno nuovo; un ri-clustering periodico (DBSCAN) consolida i gruppi. L'utente può assegnare un nome a ciascuna persona e sfogliare tutte le sue occorrenze nell'archivio.

2.7 Controllo Integrità Archivio

Accessibile dal pulsante scudo nella navbar, che riporta un badge con il numero di anomalie rilevate. Il pannello a comparsa individua due situazioni:

- File «intrusi»: presenti in data/archive ma non registrati nel database (tipicamente copiati a mano). Possono essere importati e indicizzati, oppure spostati in quarantena.
- Record «orfani»: registrati nel database ma il cui file è stato rimosso dal disco. Possono essere rimossi dall'indice.

3. Persistenza dei Dati

L'applicazione usa un'architettura ibrida composta da due database complementari: uno relazionale (fonte di verità) e uno vettoriale (indice di ricerca).

3.1 Database Relazionale — SQLite

Percorso: data/db/archive.db. Contiene le tabelle principali:

- media_items — un record per ogni file importato: id, file_path, filename, media_type, file_size, creation_date, location_lat/lon/name, width, height, duration, transcription, file_hash (UNIQUE, anti-duplicati SHA-256), processed (0=in coda, 1=ok, -1=fallito) e i tre stati di stadio stato_embedding, stato_volti, stato_trascrizione.
- media_frames — un record per frame (1 per immagine, N per video): id, media_id, frame_index, timestamp_seconds, image_path, objects (tag in JSON) e clip_embedding (BLOB float32). Nota: il nome della colonna è un'eredità dell'era CLIP; oggi contiene i vettori Qwen a 2048 dimensioni.
- faces — un record per volto: id, media_id, frame_id, crop_path, embedding (BLOB float32, 512-d FaceNet), coordinate del riquadro e person_id (raggruppamento).
- persons e impostazioni — anagrafica delle persone e stato runtime chiave/valore (es. pausa della coda).

3.2 Database Vettoriale — ChromaDB

Percorso: `chroma_db/`. Funge da indice ANN (Approximate Nearest Neighbor) per il recupero rapido dei candidati. Sono usate due collezioni distinte:

- `qwen_frames` — embedding Qwen dei frame (2048 dimensioni);
- `faces` — embedding facciali FaceNet (512 dimensioni).

La separazione è necessaria, non cosmetica: `media_frames.id` e `faces.id` sono sequenze `AUTOINCREMENT` indipendenti, quindi il frame 1 e il volto 1 condividerebbero lo stesso id. Un'unica collezione causerebbe collisioni silenziose nel recupero.

3.3 Doppia memorizzazione degli embedding (scelta progettuale)

Ogni embedding è memorizzato due volte: come BLOB float32 in SQLite e come vettore in ChromaDB. È una scelta deliberata, motivata da tre esigenze:

- Precisione del ranking. ChromaDB restituisce i candidati top-K per distanza L2, ma SQLite è la fonte di verità: sul sottoinsieme di candidati il coseno esatto viene ricalcolato in Python (`numpy.dot`) contro i BLOB, perché le soglie dell'applicazione sono espresse in coseno, non in L2.
- Resilienza. Se ChromaDB non è disponibile, la ricerca ripiega automaticamente su una scansione lineare degli embedding in SQLite: l'applicazione resta funzionante.
- Auto-riparazione. All'avvio, se una collezione ChromaDB è vuota ma SQLite contiene dati, l'indice viene ricostruito dai BLOB. È quindi sufficiente eliminare `chroma_db/` per forzarne la ricostruzione.

4. Requisiti del Sistema

4.1 Piattaforma di Sviluppo

Voce	Dettaglio
Linguaggio	Python 3.11+ (64-bit); l'installer fornisce automaticamente Python 3.13 su Windows
Framework UI	Streamlit
OS target	Windows 10/11 64-bit; supportato anche Linux/macOS
Ambiente virtuale	venv (creato automaticamente dall'installer)
Server modelli	llama.cpp / llama-server, sottoprocesso locale che serve il modello Qwen
Dipendenze di sistema	FFmpeg fornito dal pacchetto Python <code>imageio-ffmpeg</code> , nessuna installazione manuale

Tabella 2 — Piattaforma di sviluppo ed esecuzione.

4.2 Modelli di Intelligenza Artificiale

Tutti i modelli sono gratuiti, open ed eseguiti interamente in locale, così nessun contenuto privato lascia la macchina.

Modello	Ruolo
Qwen3-VL-Embedding-2 B	Embedding multimodale immagine+testo (2048-d): ricerca semantica, similarità immagine e tagging zero-shot. «Legge» anche il testo presente nelle immagini a livello di embedding (sostituisce CLIP ed EasyOCR). Servito da llama-server.
MTCNN + FaceNet	Rilevamento dei volti (MTCNN) ed embedding facciale a 512-d (FaceNet / InceptionResnetV1, VGGFace2).
Whisper (base)	Trascrizione dell'audio dei video (Speech-to-Text).

Tabella 3 — Modelli AI locali utilizzati dall'applicazione.

Il tagging zero-shot è multi-etichetta: una categoria diventa un tag se il coseno immagine↔categoria supera una soglia calibrata (nessun softmax, così le categorie non competono tra loro).

4.3 Architettura Software

Il sistema è organizzato in tre strati con separazione netta di responsabilità (Figura 3). Il flusso completo di una ricerca è illustrato in Figura 4.

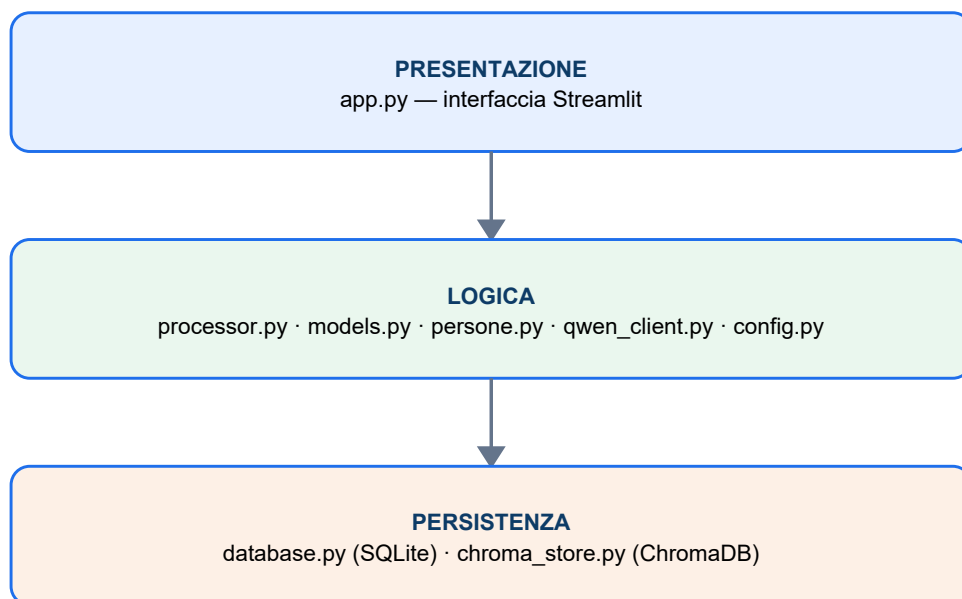


Figura 3 — Architettura software a tre strati, con separazione delle responsabilità.

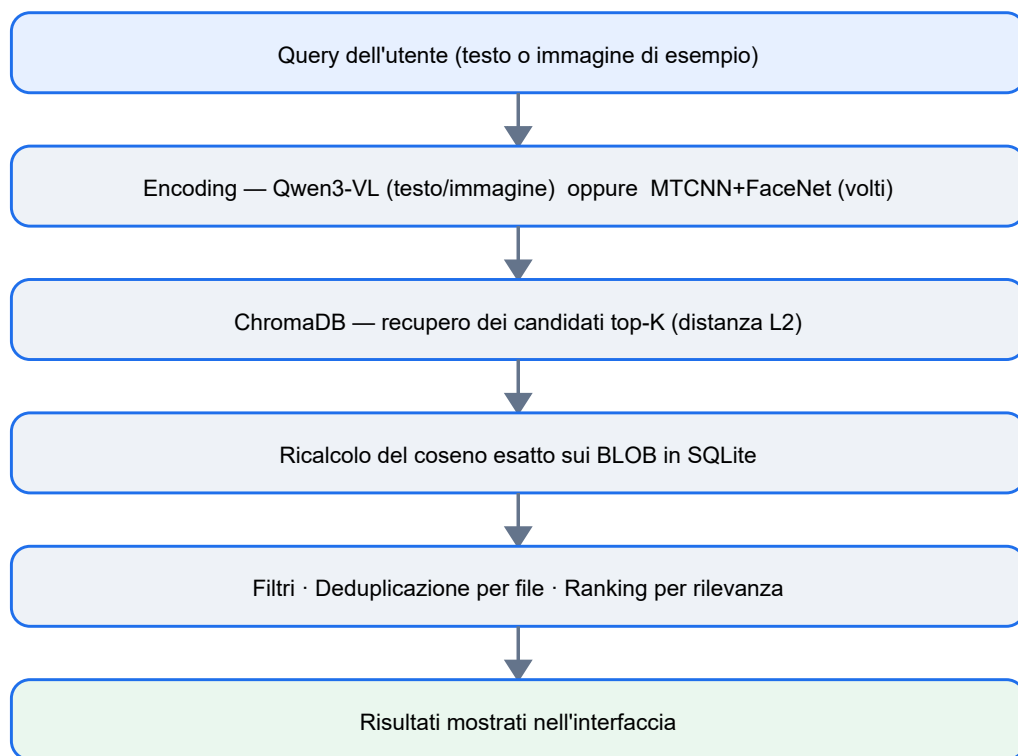


Figura 4 — Flusso di elaborazione di una ricerca, dalla query alla griglia dei risultati.

Qwen non è caricato in-process: `GestoreModelli` avvia pigramente un sottoprocesso `llama-server` che serve i file GGUF del modello su una porta locale, pilotato via HTTP da `qwen_client.py`.

4.4 Requisiti Minimi Hardware

Componente	Minimo	Consigliato
CPU	Quad-core 64-bit	8-core
RAM	8 GB	16 GB
Disco	5 GB liberi (~2,1 GB per i modelli Qwen)	20+ GB
GPU	Non richiesta (esecuzione su CPU)	NVIDIA CUDA (VRAM ≥ 4 GB)

Tabella 4 — Requisiti hardware minimi e consigliati.

L'applicazione funziona anche senza GPU: in assenza di CUDA i modelli girano su CPU (con prestazioni inferiori) e l'offload GPU di `llama-server` viene automaticamente disattivato.

5. Descrizione delle Classi e dei Moduli

Tutti i moduli si trovano in `src/`. Il grafo delle dipendenze è:

```

app.py      -> config, database, processor, models, persone
processor.py -> config, database, models, persone
database.py -> config, chroma_store
  
```

```
models.py    -> config, qwen_client
persone.py   -> config, database
```

5.1 models.py

Classe	Responsabilità
GestoreQwen	Avvio/gestione del sottoprocesso llama-server; generazione degli embedding di testo e immagine (2048-d) via qwen_client.
RiconoscitoreVolti	MTCNN per il rilevamento + FaceNet per l'embedding facciale; filtra confidenza < 0,90 e volti < 40 px.
GestoreWhisper	Whisper base; carica un WAV 16 kHz mono e disabilita fp16 su CPU.
GestoreModelli	Singleton con lazy-loading thread-safe: i modelli sono caricati solo al primo utilizzo; libera_memoria() li rilascia e termina llama-server.

Tabella 5 — Classi del modulo models.py.

5.2 processor.py

Implementa la pipeline di ingestione come coda a stadi ripartibile (non un'unica pipeline sincrona). Funzioni principali:

Funzione	Descrizione
registra_file()	Parte veloce e sincrona: dedup SHA-256, copia in data/archive (rinominata con l'hash), inserimento del record (processed=0).
avvia_lavoratore()	Avvia il thread daemon che lavora la coda (persistente su DB): riprende esattamente da dove si era interrotto dopo un riavvio.
_prepara_media()	Stadio veloce senza AI: anteprima video, EXIF/GPS, estrazione dei frame video a intervalli regolari.
_stadio_embedding()	Embedding Qwen di ogni frame + tag zero-shot multi-etichetta.
_stadio_volti()	MTCNN/FaceNet e assegnazione del volto a una persona (persone.assegna_persona).
_stadio_trascrizione()	Solo video: estrazione audio (FFmpeg) e trascrizione con Whisper.
trova_file_intrusi()	Confronto file system ↔ DB per rilevare file copiati a mano.

Tabella 6 — Funzioni principali del modulo processor.py.

5.3 chroma_store.py — Classe ChromaStore

Wrapper su chromadb.PersistentClient. Metodi principali: aggiungi_o_aggiorna() (upsert), cerca_simili() (top-K per distanza L2), elimina() e conteggio(). Ogni collezione (qwen_frames, faces) ha un indice persistente proprio.

5.4 database.py

Funzioni di accesso a SQLite con sincronizzazione automatica su ChromaDB. Gestisce la serializzazione/deserializzazione degli embedding (BLOB NumPy float32), il ricalcolo del coseno esatto sui candidati, la ricostruzione dell'indice vettoriale dai dati SQLite e il cascading delete all'eliminazione di un media.

5.5 qwen_client.py e persone.py

- qwen_client.py — client HTTP verso llama-server: pre-elaborazione delle immagini (smart_resize, codifica base64) e chiamata dell'endpoint di embedding.
- persone.py — raggruppamento dei volti in persone: assegnazione greedy al centroide più vicino e ri-clustering con DBSCAN.

5.6 config.py

Raccoglie in un unico file di testo tutti i parametri modificabili (percorsi, modelli, soglie, dispositivo hardware). Vedi la sezione 6.3.

6. Deployment

Gli script di installazione e avvio sono divisi per sistema operativo in scripts/windows/ e scripts/linux/.

6.1 Installazione (una tantum)

Windows — doppio clic su scripts\windows\install.bat (involucro che richiama install.ps1). Lo script esegue automaticamente:

- verifica/installazione di Python 3.13 (minimo richiesto: 3.11);
- creazione dell'ambiente virtuale venv/;
- rilevamento della GPU NVIDIA (via nvidia-smi, con fallback WMI) e installazione della build PyTorch corretta (CUDA o CPU-only);
- installazione delle dipendenze da requirements.txt e di facenet-pytorch con --no-deps;
- download dei modelli Qwen (~2,1 GB) e di llama-server, con verifica SHA-256 dei file.

Linux/macOS — ./scripts/linux/install.sh: stesso flusso, ma non installa Python automaticamente e rileva la GPU solo tramite nvidia-smi (build CPU di llama-server).

6.2 Avvio

Windows: doppio clic su scripts\windows\run.bat. Linux/macOS: ./scripts/linux/run.sh. Il browser si apre automaticamente su <http://localhost:8501>; per terminare, Ctrl+C nel terminale.

6.3 Configurazione

Tutti i parametri modificabili si trovano in `src/config.py`:

```
NOME_MODELLO_WHISPER      = "base"      # tiny | base | small | medium
MODALITA_DISPOSITIVO      = "auto"      # auto | cpu | cuda
INTERVALLO_FRAME_VIDEO    = 3.0         # secondi tra un frame e il successivo
SOGLIA_SIMILARITA_VOLTI   = 0.72        # soglia coseno riconoscimento volti
SOGLIA_COSENO_TAG         = 0.385       # soglia coseno per assegnare un tag
LAMBDA_PROMPT_NEGATIVO    = 0.5         # peso del prompt negativo
QWEN_NGL                  = 99          # strati Qwen su GPU (0 = forza CPU)
MASSIMO_TAG_PER_IMMAGINE  = 10
```

6.4 Controllo di Versione

Il progetto è versionato con Git (VCS distribuito), con hosting su piattaforma remota (GitHub). Il file `.gitignore` esclude le cartelle generate automaticamente e i dati locali:

```
venv/
chroma_db/
data/
models/qwen/*.gguf
__pycache__/
```

7. Riferimenti

Risorsa	Link
Streamlit	https://streamlit.io/docs
Qwen3-VL-Embedding-2B (GGUF)	https://huggingface.co/DevQuasar/Qwen.Qwen3-VL-Embedding-2B-GGUF
llama.cpp / llama-server	https://github.com/ggml-org/llama.cpp
FaceNet-PyTorch	https://github.com/timesler/facenet-pytorch
Whisper	https://github.com/openai/whisper
ChromaDB	https://www.trychroma.com
PyTorch	https://pytorch.org
OpenCV	https://docs.opencv.org

Tabella 7 — Documentazione e risorse di riferimento.